

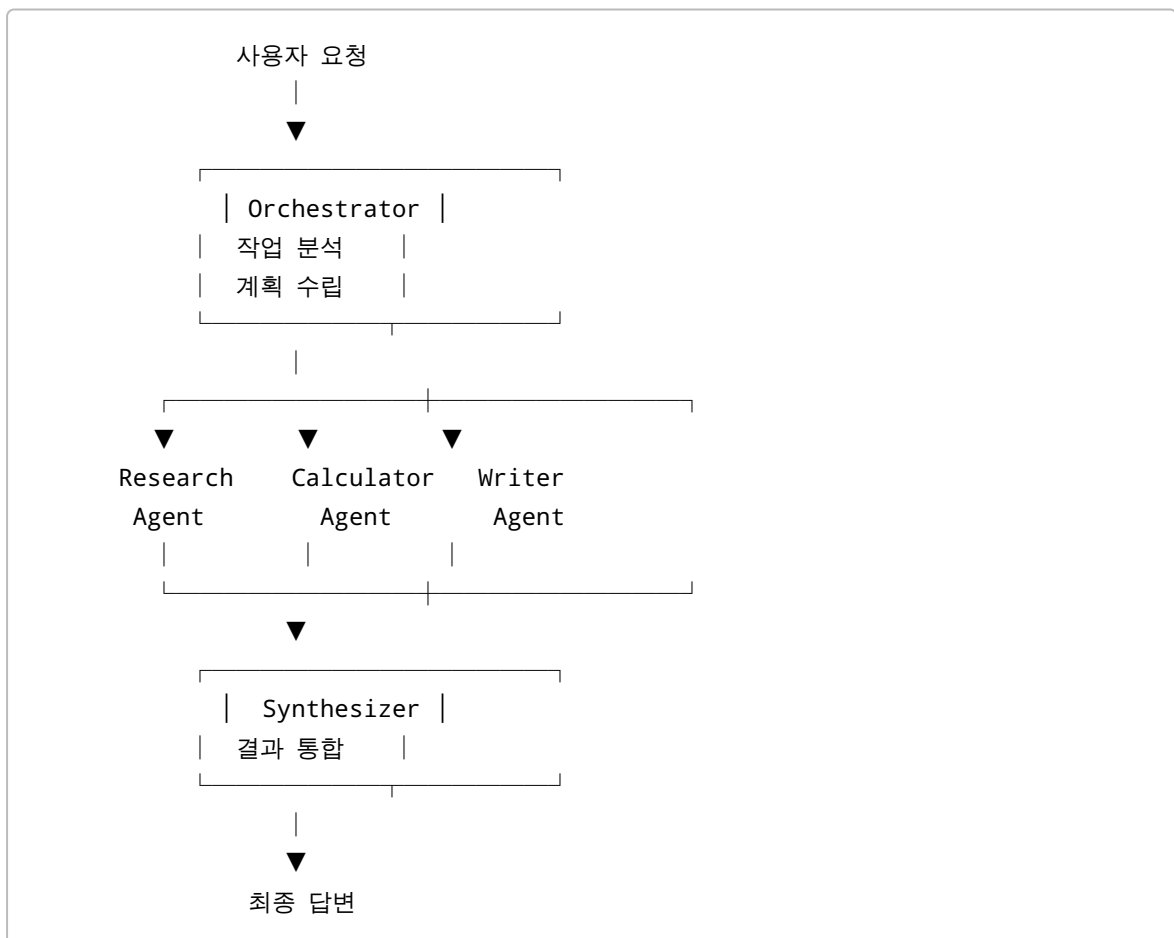
Orchestrator / Sub-agent 패턴 설계

1. Orchestrator / Sub-agent 패턴 개요

멀티에이전트 시스템은 하나의 AI Agent가 모든 작업을 수행하는 대신, 여러 Agent에게 역할을 분담하여 복잡한 문제를 해결하는 구조이다.

그중 Orchestrator / Sub-agent 패턴은 하나의 중앙 Agent가 전체 작업을 관리하고, 전문화된 여러 Sub-agent에게 필요한 작업을 위임하는 대표적인 멀티에이전트 설계 방식이다.

기본적인 구조는 다음과 같다.



핵심은 "하나의 Agent가 모든 일을 수행하는 구조"가 아니라 "전체 작업을 조정하는 Agent와 전문 작업을 수행하는 Agent를 분리하는 것"이다.

2. 왜 Orchestrator / Sub-agent 패턴이 필요한가

단일 Agent 구조에서는 하나의 Agent가 다음과 같은 여러 역할을 동시에 수행해야 한다.

```

사용자 요청
↓
문제 분석
↓
작업 계획
↓
정보 검색
↓
계산
↓
문서 작성
↓
결과 검증
↓
최종 답변

```

작업의 종류가 많아질수록 하나의 프롬프트와 Agent 로직이 복잡해진다.

예를 들어 다음과 같은 요청을 생각할 수 있다.

스타트업 A, B, C의 투자 유치 금액을 조사하고
총액을 계산한 다음
투자자에게 보낼 이메일을 작성하라.

하나의 Agent가 모두 처리할 수도 있지만 다음과 같이 역할을 분리할 수 있다.

```

Orchestrator
|
├── Research Agent
|   └── 투자 유치 금액 조사
|
├── Calculator Agent
|   └── 투자 금액 합산
|
└── Writer Agent
    └── 이메일 작성

```

이렇게 하면 각 Agent가 자신의 역할에 집중할 수 있다.

장점은 다음과 같다.

- 복잡한 문제를 작은 작업으로 분해할 수 있다.
- Agent별 책임이 명확해진다.
- Agent를 독립적으로 테스트할 수 있다.
- 특정 Agent만 다른 Agent로 교체할 수 있다.

- 필요한 Agent만 선택적으로 실행할 수 있다.
- 검색, 계산처럼 LLM이 필요하지 않은 작업을 별도의 도구로 구현할 수 있다.
- 전체 시스템의 유지보수가 쉬워진다.

3. Orchestrator란 무엇인가

Orchestrator는 멀티에이전트 시스템의 전체 작업을 조정하는 중앙 제어 역할이다.

주요 역할은 다음과 같다.

3.1 사용자 요청 분석

사용자의 자연어 요청을 분석하여 필요한 작업을 파악한다.

예:

"스타트업 A, B, C의 투자 유치 금액을 조사하고
합계를 계산한 후 투자자용 이메일을 작성해줘."

Orchestrator는 이를 다음과 같이 분석할 수 있다.

필요한 작업

1. A, B, C 투자 유치 금액 조사
2. 투자 금액 합산
3. 투자자용 이메일 작성

3.2 작업 분해

하나의 복잡한 요청을 실행 가능한 Sub-task로 분해한다.

Task

- ├ Sub-task 1
- ├ Sub-task 2
- └ Sub-task 3

각 Sub-task에는 담당 Agent와 실행 지시사항이 포함될 수 있다.

```
{  
  "agent": "research",  
  "instruction": "스타트업 A, B, C의 투자 유치 금액을 조회한다."  
}
```

3.3 실행 순서 결정

Sub-task 사이에 의존성이 존재하는 경우 실행 순서를 결정한다.

```
graph TD;
  Research --> Calculator;
  Calculator --> Writer;
```

계산 Agent는 Research 결과가 필요하고 Writer는 계산 결과가 필요하기 때문에 순차적인 실행이 필요하다.

3.4 적절한 Sub-agent 선택

모든 요청에서 모든 Agent를 실행할 필요는 없다.

예를 들어 다음 요청에서는 Writer가 필요하지 않다.

스타트업 A와 B의 투자 유치 금액 합계만 알려줘.

따라서 Orchestrator는 다음과 같이 계획할 수 있다.

```
graph TD;
  Research --> Calculator;
```

이처럼 필요한 Agent만 동적으로 선택하는 것이 Orchestrator 구조의 중요한 특징이다.

4. Sub-agent란 무엇인가

Sub-agent는 Orchestrator가 위임한 특정 작업을 수행하는 전문 실행 단위이다.

Sub-agent는 하나의 LLM일 수도 있고, 특정 기능을 수행하는 프로그램이나 Tool일 수도 있다.

예를 들어 다음과 같이 구성할 수 있다.

Sub-agent	주요 역할
Research Agent	정보 검색 및 조사
Calculator Agent	수치 계산
Writer Agent	문서 및 자연어 생성
Translator Agent	언어 번역

Sub-agent	주요 역할
Summarizer Agent	문서 요약
Validator Agent	결과 검증
SQL Agent	데이터베이스 조회
Vision Agent	이미지 분석

중요한 원칙은 하나의 Sub-agent가 너무 많은 역할을 담당하지 않도록 하는 것이다.

예를 들어 다음과 같은 Agent는 역할이 지나치게 넓다.

```

General Agent
├── 검색
├── 계산
├── 번역
├── 요약
├── 문서 작성
└── 검증
  
```

대신 다음과 같이 전문화하는 것이 좋다.

```

Research Agent
Calculator Agent
Writer Agent
Validator Agent
  
```

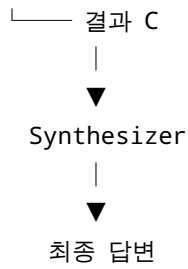
5. Synthesizer의 역할

Orchestrator가 작업을 분배하고 Sub-agent가 개별 작업을 수행하면 각각의 결과가 생성된다.

이 결과를 하나의 최종 답변으로 통합하는 역할을 Synthesizer가 담당할 수 있다.

```

Research Agent
  |
  ├── 결과 A
  |
Calculator Agent
  |
  ├── 결과 B
  |
Writer Agent
  |
  
```



Synthesizer는 다음과 같은 작업을 수행한다.

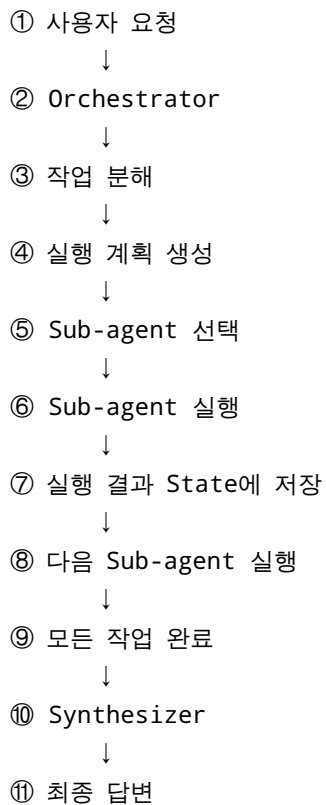
- Sub-agent 결과 수집
- 결과 간 관계 확인
- 필요한 결과 선택
- 중복 내용 제거
- 최종 응답 형식 구성

단순한 시스템에서는 Writer의 결과를 그대로 최종 답변으로 사용할 수도 있다.

반대로 여러 Agent의 결과를 종합해야 하는 시스템에서는 별도의 Synthesizer Agent를 두는 것이 효과적이다.

6. 전체 실행 흐름

Orchestrator / Sub-agent 패턴의 전체적인 실행 과정은 다음과 같다.



예를 들어 다음 요청을 처리한다고 가정한다.

스타트업 A, B, C의 투자 유치 금액 합계를 구하고
투자자에게 보낼 요약 이메일을 작성해줘.

Orchestrator가 다음 계획을 만든다.

1. Research
스타트업 A, B, C의 투자 유치 금액을 조회한다.
2. Calculator
A, B, C의 투자 유치 금액을 합산한다.
3. Writer
계산 결과를 바탕으로 투자자용 이메일을 작성한다.

실행 결과는 다음과 같다.

Research
→ A: 120억원
→ B: 35억원
→ C: 68억원

Calculator
→ $120 + 35 + 68$
→ 223억원

Writer
→ 투자자용 이메일 작성

Synthesizer
→ 최종 답변

7. Task Queue를 이용한 작업 관리

Orchestrator가 생성한 작업을 Queue 형태로 관리하면 전체 시스템을 단순하게 설계할 수 있다.

예를 들어 다음과 같은 State를 사용할 수 있다.

```
class AgentState(TypedDict):  
    request: str  
    plan: List[Dict[str, str]]
```

```
pending: List[Dict[str, str]]
completed: List[Dict[str, str]]
final_answer: str
```

각 필드의 역할은 다음과 같다.

필드	의미
request	사용자의 원본 요청
plan	Orchestrator가 생성한 전체 실행 계획
pending	아직 실행하지 않은 작업
completed	실행이 완료된 작업과 결과
final_answer	최종 답변

예를 들어 처음에는 다음과 같이 구성된다.

```
pending

1. research
2. calculator
3. writer
```

Research Agent가 실행되면:

```
pending

1. calculator
2. writer

completed

1. research
```

Calculator가 실행되면:

```
pending

1. writer

completed
```


1. research
2. calculator

Writer까지 완료되면:

```
pending

없음

completed

1. research
2. calculator
3. writer
```

이후 Synthesizer가 최종 답변을 생성한다.

8. Agent Routing

멀티에이전트 시스템에서 중요한 기능 중 하나가 Routing이다.

Routing은 현재 작업을 어떤 Agent에게 전달할 것인지 결정하는 과정이다.

예를 들어 다음과 같이 Agent를 구성한다.

```
research
calculator
writer
```

현재 pending 작업이 다음과 같다고 가정한다.

```
{
  "agent": "calculator",
  "instruction": "A, B, C의 투자 금액을 합산한다."
}
```

Router는 `agent` 값을 확인하고 Calculator Agent로 작업을 전달한다.

개념적으로 다음과 같다.

```
def route_next(state):
    if not state["pending"]:
```

```

return "synthesizer"

return state["pending"][0]["agent"]

```

즉,

```

pending 없음
  → Synthesizer

agent = research
  → Research Agent

agent = calculator
  → Calculator Agent

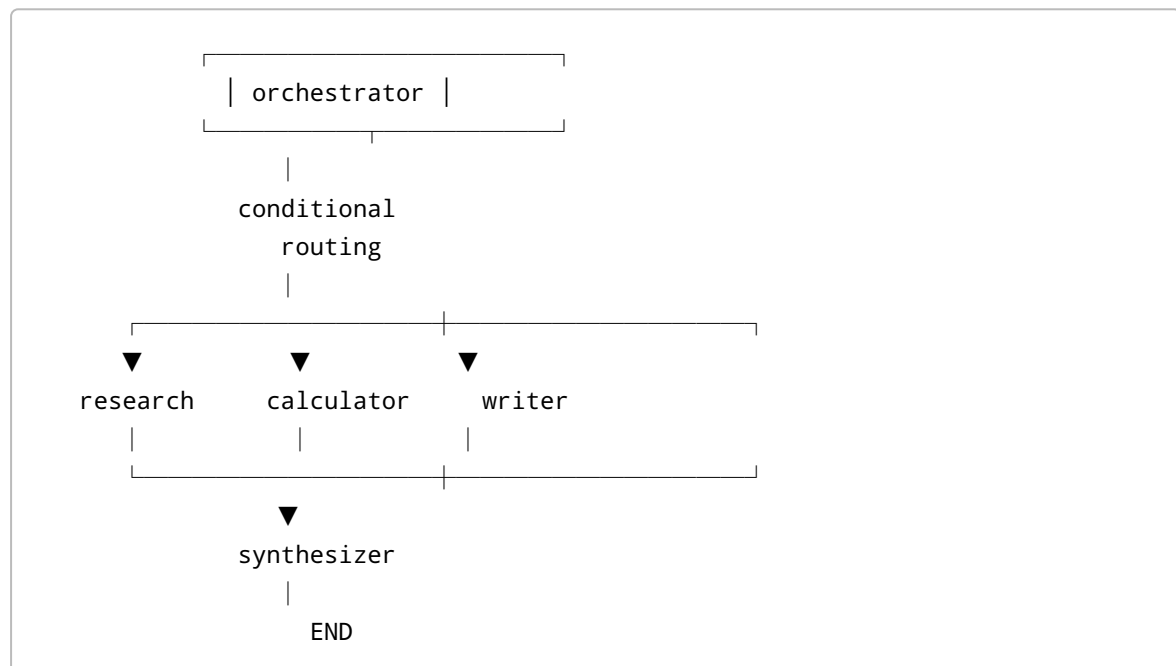
agent = writer
  → Writer Agent

```

이러한 구조를 사용하면 Agent가 추가되더라도 전체 Routing 로직을 크게 변경할 필요가 없다.

9. LangGraph를 이용한 구현 구조

LangGraph에서는 각각의 Agent를 Node로 표현할 수 있다.



각 Node는 State를 입력으로 받고 변경된 State를 반환한다.

```
def research_node(state):
    ...
    return {
        "pending": ...,
        "completed": ...
    }
```

Calculator도 동일한 방식으로 동작한다.

```
def calculator_node(state):
    ...
    return {
        "pending": ...,
        "completed": ...
    }
```

Writer는 LLM을 호출하여 자연어 결과를 생성한다.

```
def writer_node(state):
    ...
    response = llm.invoke(...)
    ...
    return {
        "pending": ...,
        "completed": ...
    }
```

이러한 구조에서 각 Node는 자신의 역할에 집중하고 State를 통해 작업 결과를 공유한다.

10. 모든 Sub-agent가 LLM일 필요는 없다

멀티에이전트 시스템을 설계할 때 중요한 원칙이다.

Sub-agent라는 이름 때문에 모든 작업을 LLM으로 구현할 필요는 없다.

예를 들어 계산은 일반 Python 코드로 수행하는 것이 더 적절하다.

```
numbers = [120, 35, 68]
total = sum(numbers)
```

검색 결과에서 숫자를 추출하는 것도 일반 프로그램으로 처리할 수 있다.

```
numbers = re.findall(r"(\d+)억원", source_text)
```

반면 자연스러운 이메일 작성은 LLM이 적합하다.

Research

→ 일반 검색 또는 Database

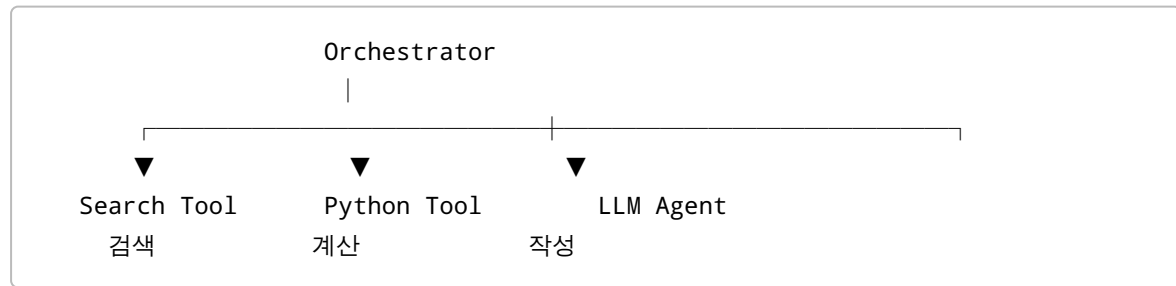
Calculator

→ Python 함수

Writer

→ LLM

따라서 실제 시스템에서는 다음과 같이 구성하는 것이 효율적이다.



이 방식은 불필요한 LLM 호출을 줄이고 시스템의 속도와 안정성을 높이는 데 유리하다.

11. Structured Output을 이용한 계획 생성

Orchestrator가 자유로운 자연어로 계획을 생성하도록 하면 이후 프로그램에서 결과를 처리하기 어렵다.

따라서 계획을 구조화된 데이터로 생성하는 것이 좋다.

예:

```
class SubTask(BaseModel):
    agent: Literal["research", "calculator", "writer"]
    instruction: str

class Plan(BaseModel):
    subtasks: List[SubTask]
```

LLM은 다음과 같은 구조를 반환한다.

```

Plan
└─ subtasks
    │   └─ agent: research
    │       instruction: 투자 유치 금액 조회
    │
    │   └─ agent: calculator
    │       instruction: 투자 유치 금액 합산
    │
    └─ agent: writer
        instruction: 투자자용 이메일 작성

```

이렇게 하면 프로그램이 `agent` 값을 이용하여 자동으로 Routing할 수 있다.

12. 동적 Agent 선택

Orchestrator / Sub-agent 구조의 중요한 특징은 모든 요청에서 동일한 Agent를 실행하지 않는다는 것이다.

예를 들어 요청 1:

A, B, C의 투자 유치 금액을 조사하고
총액을 계산해서 이메일을 작성해줘.

실행 계획:

```

Research
  ↓
Calculator
  ↓
Writer

```

요청 2:

A와 B의 투자 유치 금액 합계만 알려줘.

실행 계획:

```

Research
  ↓
Calculator

```

요청 3:

다음 내용을 투자자용 이메일로 작성해줘.

실행 계획:

Writer

즉, Orchestrator는 요청에 따라 필요한 Agent만 선택한다.

이러한 동적 선택은 멀티에이전트 시스템의 비용과 실행 시간을 줄이는 데 중요한 요소이다.

13. Orchestrator와 Sub-agent의 책임 분리

각 구성요소의 책임을 명확하게 정의해야 한다.

구성요소	책임
Orchestrator	문제 분석, 작업 분해, 실행 계획
Sub-agent	자신에게 할당된 작업 실행
State	작업 상태와 결과 전달
Router	다음 실행 Agent 결정
Synthesizer	결과 통합 및 최종 답변

특히 Orchestrator가 직접 모든 작업을 수행하지 않도록 하는 것이 중요하다.

좋은 구조:

Orchestrator

→ "무엇을 할 것인가" 결정

Sub-agent

→ "실제로 어떻게 수행할 것인가" 처리

14. Orchestrator 설계 원칙

원칙 1. 역할을 명확하게 분리한다

각 Agent는 가능한 한 하나의 전문적인 책임을 갖도록 설계한다.

Research Agent

→ 정보 조사

Calculator Agent

→ 계산

Writer Agent

→ 문서 작성

원칙 2. 작업 단위를 명확하게 정의한다

Sub-task에는 단순히 Agent 이름만 지정하는 것이 아니라 구체적인 Instruction을 전달하는 것이 좋다.

좋지 않은 예:

calculator

좋은 예:

스타트업 A, B, C의 투자 유치 금액을 합산하라.

원칙 3. Agent 간 의존성을 고려한다

다음 작업에 이전 작업 결과가 필요한지 판단해야 한다.

Research

↓

Calculator

이 구조에서는 Calculator가 Research보다 먼저 실행될 수 없다.

원칙 4. 결정적인 작업은 코드나 Tool로 처리한다

계산, 데이터 형식 변환 등 정확성이 중요한 작업은 가능하면 LLM보다 프로그램이나 Tool을 사용하는 것이 좋다.

원칙 5. State를 통해 결과를 전달한다

Agent끼리 직접 복잡하게 연결하기보다는 공통 State를 이용하여 실행 결과를 관리하면 구조가 단순해진다.

15. Orchestrator 패턴의 장점

15.1 관심사 분리

각 Agent가 하나의 역할에 집중할 수 있다.

15.2 유지보수성

Writer Agent를 변경해도 Research Agent는 수정할 필요가 없다.

15.3 확장성

새로운 Agent를 추가하기 쉽다.

예:

```
Research
Calculator
Writer
Translator
Validator
```

15.4 선택적 실행

요청에 필요한 Agent만 실행할 수 있다.

15.5 테스트 용이성

각 Agent를 독립적으로 테스트할 수 있다.

예:

```
Research Agent 테스트
    ↓
Calculator Agent 테스트
    ↓
Writer Agent 테스트
```

16. Orchestrator 패턴의 한계

장점만 있는 것은 아니다.

16.1 중앙 Orchestrator 의존성

모든 작업이 Orchestrator를 거치므로 Orchestrator의 계획 오류가 전체 시스템에 영향을 줄 수 있다.

16.2 복잡한 계획 생성

작업이 복잡해질수록 올바른 Sub-task 분해가 어려워질 수 있다.

16.3 State 관리 복잡성

여러 Agent가 실행되면 State에 저장해야 하는 정보가 많아진다.

16.4 실행 비용 증가

여러 Agent가 LLM을 호출하면 단일 Agent보다 API 호출량이 증가할 수 있다.

16.5 불필요한 Agent 호출

Orchestrator가 잘못된 계획을 생성하면 필요하지 않은 Agent가 실행될 수 있다.

따라서 실제 시스템에서는 Agent 수를 무조건 늘리는 것이 아니라 작업 복잡도와 비용을 고려해야 한다.

17. 단일 Agent와 비교

항목	단일 Agent	Orchestrator / Sub-agent
구조	단순	복잡
구현 난이도	낮음	높음
역할 분리	낮음	높음
확장성	제한적	높음
유지보수	어려워질 수 있음	상대적으로 용이
전문화	낮음	높음
작업 분배	없음	Orchestrator가 수행
병렬 처리	제한적	설계에 따라 가능
시스템 비용	상대적으로 낮음	증가 가능
복잡한 업무	한계 존재	적합

단순한 질의응답 시스템에서는 단일 Agent가 적합할 수 있다.

반면 검색, 분석, 계산, 작성 등 여러 종류의 작업을 포함하는 복잡한 업무에서는 Orchestrator / Sub-agent 구조가 적합하다.

18. 실습 예제

다음과 같은 요구사항을 구현한다고 가정한다.

스타트업 A, B, C의 투자 유치 금액 합계를 구하고,
이를 바탕으로 투자자에게 보낼 요약 이메일을 작성한다.

Step 1. Orchestrator

요청을 다음과 같이 분해한다.

1. research
A, B, C의 투자 유치 금액 조회
2. calculator
투자 유치 금액 합산
3. writer
투자자용 이메일 작성

Step 2. Research Agent

예시 결과:

A: 120억원
B: 35억원
C: 68억원

Step 3. Calculator Agent

$120 + 35 + 68$
= 223억원

Step 4. Writer Agent

Research와 Calculator의 결과를 전달받아 이메일을 작성한다.

안녕하세요, 투자자님.

스타트업 A, B, C의 투자 유치 금액을 분석한 결과
총 투자 유치 금액은 223억원으로 확인되었습니다.

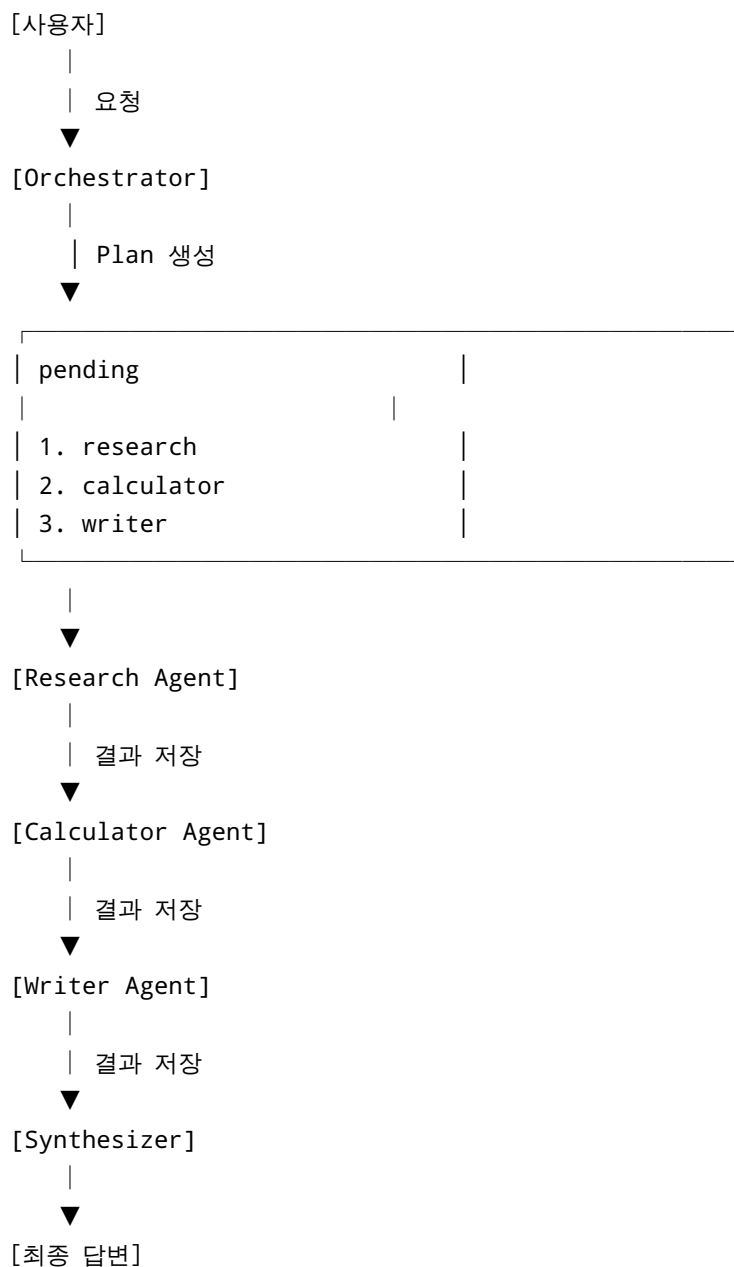
감사합니다.

Step 5. Synthesizer

각 Agent의 결과를 확인하고 최종 답변을 구성한다.

19. 실행 흐름 정리

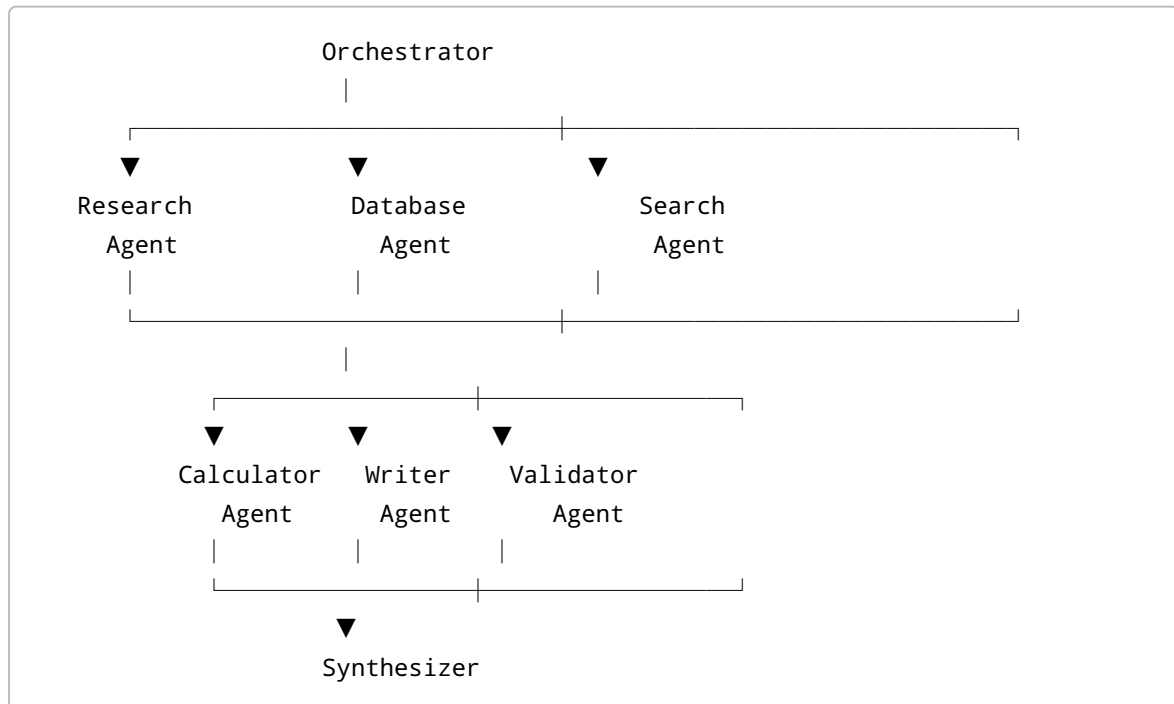
실제 실행 과정은 다음과 같이 이해할 수 있다.



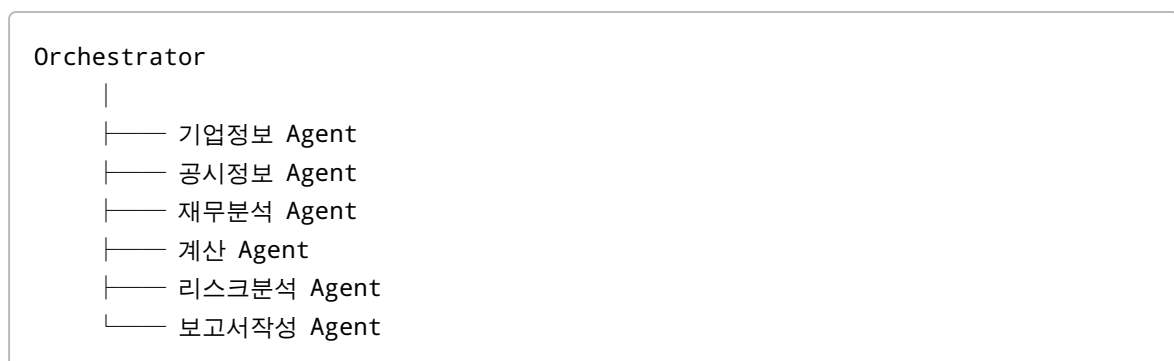
핵심은 `pending` 과 `completed` 상태를 이용하여 작업의 진행 상황을 관리하는 것이다.

20. 확장 설계

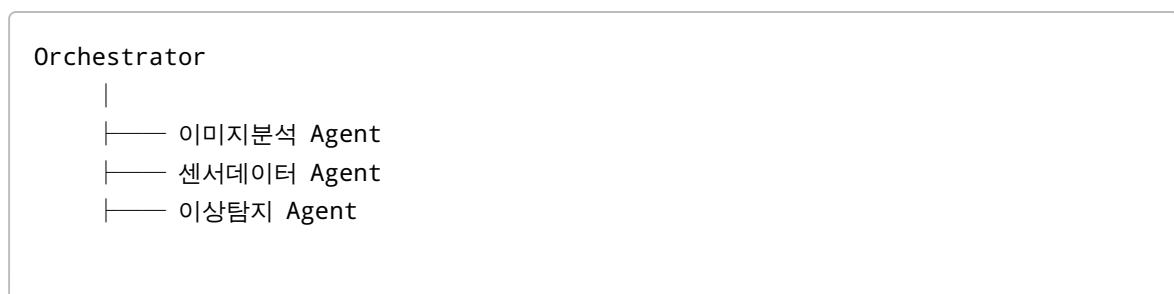
기본 구조에서는 세 개의 Agent를 사용하지만 실제 시스템에서는 다양한 전문 Agent를 추가할 수 있다.



예를 들어 금융 AI 시스템이라면 다음과 같이 구성할 수 있다.



스마트팩토리 시스템이라면 다음과 같이 구성할 수 있다.



└── 원인분석 Agent
└── 보고서작성 Agent

21. 실무 설계 시 고려사항

Orchestrator / Sub-agent 시스템을 실제 서비스로 구현할 때는 다음 사항을 고려해야 한다.

Agent 역할 정의

각 Agent가 어떤 작업을 담당하는지 명확하게 정의해야 한다.

입력과 출력 정의

Agent의 입력과 출력 형식을 명확하게 정의해야 한다.

Research Agent
Input → 검색 대상
Output → 검색 결과

Calculator Agent
Input → 숫자 데이터
Output → 계산 결과

실패 처리

Sub-agent가 실패했을 때 재시도하거나 다른 Agent로 전달하는 구조를 고려해야 한다.

Research
├── 성공 → Calculator
└── 실패 → Retry / Alternative Agent

결과 검증

중요한 업무에서는 Validator Agent를 추가할 수 있다.

Research
↓
Analysis
↓
Validator

↓
Writer

비용 관리

모든 작업을 LLM으로 처리하지 않고 가능한 작업은 Tool이나 일반 코드로 처리한다.

실행 로그

각 Agent의 실행 결과와 오류를 기록하면 시스템의 문제를 추적하기 쉽다.

22. 핵심 개념 정리

Orchestrator / Sub-agent 패턴은 복잡한 작업을 여러 전문 Agent에게 분배하여 처리하는 멀티에이전트 설계 패턴이다.

핵심 구성요소는 다음과 같다.

Orchestrator

→ 전체 작업을 분석하고 계획한다.

Sub-agent

→ 특정 작업을 전문적으로 수행한다.

State

→ 작업 상태와 결과를 전달한다.

Router

→ 다음에 실행할 Agent를 결정한다.

Synthesizer

→ 여러 결과를 종합하여 최종 답변을 생성한다.

전체 구조를 한 문장으로 정리하면 다음과 같다.

Orchestrator가 "무엇을, 누구에게, 어떤 순서로 수행할 것인지" 결정하고,
Sub-agent가 실제 작업을 수행하며,
Synthesizer가 그 결과를 하나의 최종 답변으로 통합하는 구조이다.

특히 중요한 설계 원칙은 "모든 Agent를 LLM으로 만들 필요가 없다"는 것이다.

검색, 계산, 데이터 처리처럼 결정적으로 수행할 수 있는 작업은 Tool 또는 일반 프로그램으로 구현하고, 자연어 이해와 생성처럼 LLM이 필요한 부분에만 LLM Agent를 사용하는 것이 효율적이다.

이러한 방식으로 설계하면 멀티에이전트 시스템의 역할 분담, 확장성, 테스트 용이성 및 유지보수성을 높일 수 있다.